

MA3: Multiprocessing, git, and LINUX

This assignment primarily concerns parallel programming, higher order functions, and basic usage of `git` and LINUX. The assignment consists of two major parts:

1. Parallel programming in Python

- Use Monte Carlo to approximate π and the volume of a d -dimensional hypersphere
- Comprehension, Lambda functions, `map`, `reduce`, `filter`, and `zip`
- Plotting with `matplotlib`
- Faster code using Numba
- Parallel programming in Python

2. `git` och LINUX

- Upload your solution to `git`
- Test your parallel code on IT dept LINUX machines

This assignment is more focused on testing various techniques, relative to the earlier ones. Getting all technologies to work properly might take a while, and not be directly related to writing the code. Make sure to ask for help if you get stuck, but you should also be aware that this is part of the intended assignment. Writing code is rarely done in a vacuum, but rather in complex environments where various components interact.

1. The exercises for the module can be found in the yellow frames below. There are **4 mandatory exercises** in this module.
2. When you have completed them, present the solutions to the teaching team.
3. Then, upload all scripts together with figures in a zip file to Studium.
4. To learn another way for accelerating your code, namely **integrating C++**, you are welcome to the first of our “Practice more” exercises in the Exam section.
5. As usual, each exercise has its own test file and there is a file `tests.sh/tests.bat` to run all tests together. The included file `HighOrderFunctionChecker.py` is required for these tests.
6. You will see that all results in this module have a random nature. That is, you may fail tests even if your implementation is correct, before you set a fixed *random seed*.

Important note:

You may collaborate with other students, search the Internet, and get a general overview using AI tools, but you must write and be able to explain the code you actually present. You may not copy code, either from other students, automated tools, or from the Internet, except from those places explicitly pointed out in this module.

Changing variable names and similar superficial modifications does not count as writing your own code. Also, aiding somebody to cheat (for example, if somebody hands in your code as their own) is not permitted and will be reported.

Since these assignments are included as part of the course examination, we are obliged to report blatant violations of these rules to the disciplinary board.

All course material that we provide is copyrighted and may not be spread.

1 Parallel programming in Python

In this part of the assignment, you will first write a program that uses Monte Carlo methodology to compute an **approximation of π** . Then you will modify it to approximate the **volume of a d -dimensional hypersphere**. Finally, you will modify that code using several techniques to make it run faster and execute it on the department's Linux servers.

1.1 Monte Carlo approximation of π

Monte Carlo methods (https://en.wikipedia.org/wiki/Monte_Carlo_method) belong to a large and important class of methods and are used within many different algorithms to solve problems numerically; most importantly in optimization, integration, and probability theory.

They utilize random tests to approximate the actual value of an underlying property. This can be very useful and important for multidimensional problems where, for example, integration in all dimensions is impossible to do exactly with regular methods.

In this part of the assignment you will write a program that uses Monte Carlo to estimate the constant $\pi = 3.14159\dots$

Figure 1 shows, in red, a circle with radius r and area $A_c = \pi r^2$. It is placed in a blue square, with sides $2r$, that has area $A_s = (2r)^2 = 4r^2$.

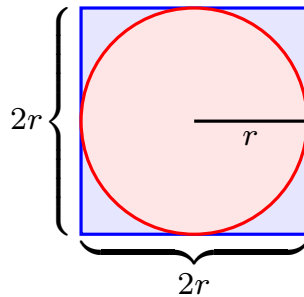


Figure 1: Circle with radius r inscribed in a square with sides $2r$.

If you divide the area A_c by A_s , you have,

$$\frac{A_c}{A_s} = \frac{\pi r^2}{4r^2} = \frac{\pi}{4},$$

or

$$\pi = 4 \frac{A_c}{A_s}.$$

Now assume that $r = 1$ and that the center of the circle is at the origin, $(0,0)$. Now create n uniformly distributed random points with coordinates $(x, y) \in [-1, 1] \times [-1, 1]$, within the square, and classify these into n_c points that lie inside the circle and n_s that lie outside the circle ($n = n_c + n_s$). One can then approximate π with

$$\pi \approx 4 \frac{n_c}{n}.$$

because the ratio n_c/n will approximate A_c/A_s , and when $n \rightarrow \infty$ we get identity.

In Figure 2 four examples are shown, for $n = \{50, 100, 200, 400\}$. The approximations of π are then $\pi \approx \{2.8, 3.16, 2.96, 2.97\}$. Note that it can happen that one sometimes gets a

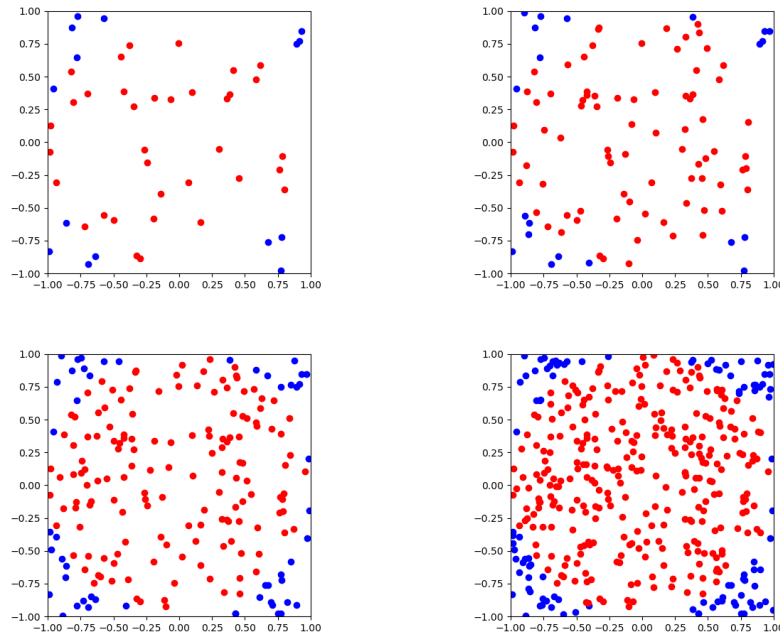


Figure 2: Approximation of π using Monte Carlo. Top: $n = 50, \pi \approx 2.8$ and $n = 100, \pi \approx 3.16$. Bottom: $n = 200, \pi \approx 2.96$ and $n = 400, \pi \approx 2.97$.

better approximation of π for a smaller n compared to a different run with a larger n , but as $n \rightarrow \infty$ the approximation comes arbitrarily close to π .

How do you decide if a point is inside the circle? In this simple case, when $r = 1$ and the center is at the origin, one can simply test if $x^2 + y^2 \leq 1$ (equivalent to $\sqrt{x^2 + y^2} \leq 1$). If one had a more complicated object, one could divide the domain into a mesh (for example a square grid or a triangulation), where each point would be either inside or outside the object.

Now write a program that has n , the number of random points that should be generated, as an argument and produces the following output:

1. Print the number of points n .
2. Print and return the approximation of $\pi \approx 4n_c/n$.
3. Produce a **png** file that shows all points inside the circle as red dots and points outside the circle as blue dots (like in Figure 2).

Tip: Use `random.uniform()` or `random.random()` to create the n random points. Also, use `matplotlib.pyplot` to create the figures. (Note: Do not take screenshots. Rather, write the figure to a file in your code.)

Exercise 1: Approximating π

Compute approximations of π for $n = \{1000, 10000, 100000\}$. Produce figures for these values of n .

1.2 Approximate the volume of a d -dimensional hypersphere

- In this part you will use higher order functions. See separate pdf in STUDIUM (Module M3) or lecture notes for an introduction. You must use at least three of the following concepts/functionalities:
 - Comprehension
 - Lambda functions
 - `map()`
 - `functools.reduce()`
 - `filter()`
 - `zip()`

in this part of the assignment.

In this part of MA3 you should write a program that uses a Monte Carlo simulation to approximate the volume $V_d(r)$ for a d -dimensional hypersphere of radius r . A hypersphere is a generalization of the circle and sphere to higher dimensions than two and three. The function `sphere_volume()` should have two arguments, n which is the number of random points to generate, and d which is the number of dimensions. One can assume $r = 1$ and the center of the hypersphere is the origin. However, one may also include r as the third parameter.

The test to see if a point is inside the hypersphere is in this case,

$$x_1^2 + x_2^2 + \dots + x_d^2 \leq 1.$$

The formula for the volume $V_d(r)$ (https://en.wikipedia.org/wiki/Volume_of_an_n-ball), to verify your code, is

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)} r^d, \quad V_d(1) = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)}$$

where Γ is the Gamma function (https://en.wikipedia.org/wiki/Gamma_function). The Gamma function already exists in Python (in the module `math`) and you do not have to implement it yourself. To solve this assignment you may use `numpy` if you want to.

Exercise 2: The volume of a hypersphere

- Write a function `sphere_volume(n,d)` that computes the approximation for $V_d(1)$ by generating n random points. Use at least three concepts from higher order functions/list comprehension.
- Write a function `hypersphere_exact` that calculates the exact volume. Compare (print) the results produced by the exact formula and the approximation for $(n, d) = (100000, 2)$ and $(n, d) = (100000, 11)$.

1.3 Numba

While Python in general is slow, especially for long loops, it is also easy to read. The main reason for Python being slow is that Python is *interpreted* in its default implementation. The `python` program reads a representation of the Python code each time and actually

executes it, similar to what we will be doing with mathematical expressions in MA4. The other option is a *compiled* language, where the program source code has been pre-translated to the actual machine code that the computer can execute, with no extra layer.

If a piece of code cannot be understood, it cannot be maintained easily and will quickly become obsolete. Especially for scientific uses, being able to peer-review code and easily try different things is essential, meaning readability is of particular value. Sometimes, it still makes sense to implement or interact with code written in other languages, such as C++. However, there are also special libraries to try to get the performance of other languages, while not really leaving Python, doing away with “the two language problem”, where users would need to be proficient in both.

One such library is *Numba*, which you will use to speed up your Python code. In the extra practice material, you can also compare it to writing an implementation in C++. Numba is a so-called “Just-In-Time” (JIT) compiler. Essentially, it compiles a piece of code at run time (as opposed to C++, which is compiled before execution), meaning the first time code is run, it is converted to fast machine code. This is particularly useful on loops, where many executions of the same piece of code are conducted. The one-time compilation will greatly accelerate performance of the following executions.

In order to be able to create an efficient machine code version, Numba needs to make a few assumptions about the code, e.g. regarding data types (like many traditionally compiled languages, as opposed to generic Python that uses duck typing). We consider an example:

```
1  # Example of a function with changing types
2  def maxOfList():
3      result = 'Nothing yet'
4      lst = [1, 8, 3, 9, 14]
5      for x in lst:
6          if result == 'Nothing yet':
7              result = x
8          elif x > result:
9              result = x
10     return result
```

This code would run perfectly fine in Python, but cannot run with Numba. Can you guess why? It has to do with the fact that the variable `result` is of different types in different parts of the code; first a string (`str`) on line 3, then an integer (`int`) on lines 7 and 9. Hence, the comparison on line 8, `>`, is not well defined. It might end up comparing an integer to a string. Numba analyses the whole function during the compilation process to find a fast way to run it, and we have here an ambiguity that is not acceptable for Numba.

Numba is not as strict as C++, which has very specific rules regarding safe type conversions, but much more strict than standard Python. Preparing the code with type hinting and checking enabled can help you understand what might be problematic once you transition to Numba.

For the interested, you have here some examples of cases where you need to make some extra effort for Numba to work well and give faster code <https://numba.pydata.org/numba-doc/latest/user/troubleshoot.html>. However, your task using Numba is so simple you will not have to worry about this.

1.3.1 Setup and function decorators

Numba is an extra library that isn't part of Python itself. To use it, you therefore first need to install the Numba library. This is easily done using `pip`, similarly to installing

`matplotlib` or `numpy`. To install, simply open a command window (not in Python) and type:

```
$ pip install numba
```

If this doesn't work, you might also try `python -m pip install numba`. It's possible that you get a warning about how installing a package like this might mess up your configuration. There are various ways in Python to group packages in separate environments, but for our purposes we will simply use a flag to ignore the warning.

```
$ pip install --break-system-packages numba
```

Sometimes, the flag `-user`, to tell `pip` to explicitly only install the package for your user account, makes a difference.

On at least some of the department Linux machines, there is already a version of the package `coverage`, which Numba depends on, but Numba needs a newer version. If you get an error message mentioning `coverage` when importing Numba, you can run another command:

```
$ pip install coverage --user --break-system-packages --force
```

To apply Numba to a Python function, you will use a *function decorator*. You can read more about these online, and we've briefly shown some in lectures, but essentially these are functions that modify another function in some way (in our case: speed it up through compilation).

The syntax is adding `@decoratorName` above a function definition. So to JIT compile a function using Numba, you simply add

```
@njit
```

above your function definition. Note that you must have imported Numba to your program, just like any other library:

```
from numba import njit
```

Example:

```
1 @njit
2 def someFunction(x):
3     ...
```

Numba will then JIT compile the decorated function into machine code the first time it is called.

Exercise 3: Numba versus conventional Python

- Run the function `sphere_volume` with values $(n, d) = (10^6, 11)$ in a sequential manner using conventional Python. We simply want to estimate the volume using a large n value.
- Measure the time usage of this function call three times within the same program. Compare the timings for the first, second, and third call.
- Create a JIT-compiled version of the function using Numba, and compare the performance between the first, second, and third call for this version. Also compare these values to the previous version. Do you see the same patterns from both versions? Which one wins?

Some comments:

- Numba speeds up actual Python code. It doesn't speed up many libraries that are already implemented as compiled code, such as Numpy.
- Python is a very flexible language, especially when combined with higher-order functions. If Numba struggles with your code, try to write it as basic as possible. Numba might perform best if you use explicit loops.
- If you struggle to get Numba to run on your own computer, you can try to transition to the department Linux servers instead, which we cover later in the assignment.

1.4 Parallel programming in Python

Read the separate pdf in STUDIUM (Module M3) for an introduction to parallel programming in Python.

The goal is to make a single run of your code execute in parallel over multiple CPU cores.

Exercise 4: Parallel versus sequential

- Run the original `sphere_volume` function with values $(n, d) = (10^6, 11)$ in a sequential manner again.
- Write a function `sphere_volume_parallel` that speeds up the original (non-Numba) `sphere_volume` using parallel programming. Compare the time between the two approaches (sequential and parallel). If you want to, you can even try to make a parallel Numba version.
- Run your code on the Linux servers provided by IT Department. See more information about the servers below. Was the parallel code faster when run on servers, compared to your local computer? By how much?

2 git and a bit of Linux

This exercise should be tested on the Linux machines of the IT department. We expect you to use `git` there in order to get your files and to run the code on these machines. You could also do all the parts locally on your own machine, but in the oral presentation you have to show that you can do it on the Linux machines and use `git`.

Linux is a free open-source operating system (<https://en.wikipedia.org/wiki/Linux>) that is extensively used (including as the basis for Android). You will learn how to use some basic features.

This is useful for example if you ever need to set up servers (databases, web servers, etc.) or run programs in the cloud (with virtual machines). Both of these tasks are common in data engineering and data science.

2.1 IT department Linux machines

Choose one of the following Linux machines:

```
gullviva.it.uu.se
tussilago.it.uu.se
vitsippa.it.uu.se
enequist.it.uu.se
```

There are more machines, see a more complete list at <https://www.it.uu.se/datordrift/maskinpark/linux.html>. For some of these, you can also use a graphical client called [ThinLinc](#), but that isn't the core purpose of this exercise.

Note that you can freely switch between the different servers. All files are stored on a central filesystem. Thus, they are available in an identical way on all machines. You can also log in to multiple sessions at the same time. **NB: a/ these machines are old and can be slow; b/ if you have issues connecting, let us know immediately.**

The machines run the Ubuntu distribution of Linux. You don't have administrator (root) access, so you can't (and don't need to) use a package manager to install any programs on the system, except for using [pip](#).

To access a Linux machine you need to use a terminal program on your own machine (commands are run by writing them). You will use SSH (Secure Shell) to login to the Linux machines.

Below is a list of some terminal programs (the recommended ones are highlighted in red). You need to use some terminal program on your local computer.

Your OS	
Linux	xterm (preinstalled) (https://www.tecmint.com/linux-terminal-emulators/)
macOS	Terminal (preinstalled in /Applications/Utilities) iTerm2 (https://iterm2.com/)
Windows	builtin Powershell WSL2 (Windows Subsystem for Linux, search Microsoft Store) PuTTY (https://www.putty.org/) bitwise (https://www.bitwise.com/)

WSL is a way to run (almost) any Linux program within Windows. That can be highly useful in its own right, but it can also take some time to set up properly.

To login write (do not write the \$, when it is in the beginning of a line it only indicates that you have a terminal started, it might be different on your system as well) in the terminal:

```
$ ssh abcd1234@gullviva.it.uu.se
```

Here [abcd1234](#) is your UU account (same as you use to login to STUDIUM) and you should use your password A (the same password you use to login to STUDIUM, not 'A') when asked for it (once you press return/enter). [gullviva.it.uu.se](#) is the hostname you chose in the list of linux machines. If you use e.g. PuTTY or Bitwise you will have to enter this information in a GUI (username: [abcd1234](#) and hostname: [gullviva.it.uu.se](#)).

Here is a collection of some useful linux commands:

<code>ls</code>	List files in current directory (<code>ls -la</code> to show more information and hidden files)
<code>pwd</code>	Show where you are (print working directory)
<code>cd abc</code>	Go into a directory named <code>abc</code>
<code>cd ..</code>	Go up one step in the file system (e.g., <code>/</code> is the root directory, <code>/home</code> contains home directories and <code>/home/abcd1234</code> is the home directory of user <code>abcd1234</code>)
<code>cd</code>	Go to your home directory (where your personal files are, and this is where you are when you first log in)
<code>mkdir abc</code>	Create a directory called <code>abc</code>
<code>rm abc</code>	Remove a file called <code>abc</code>
<code>rm -rf abc</code>	Remove a file or directory called <code>abc</code> , including all files and subdirectories if a directory (more dangerous, be careful!)
<code>nano hej.txt</code>	Edit a file <code>hej.txt</code> with the editor <code>nano</code> (there are many different editors). Leave nano with <code>ctrl-x</code>
<code>python test.py</code>	Run the Python code in the file <code>test.py</code> . On some very old systems you might need to write <code>python3</code> , as just <code>python</code> would launch the older Python 2.x instead.

A tutorial with an introduction to Linux

<https://maker.pro/linux/tutorial/basic-linux-commands-for-beginners>

To execute a Python file `hello.py`, you run it as

```
$ python hello.py
```

2.2 git

`git` is used to manage source code, or files in general. We refer to the lecture material. If `git` is not installed on your local computer, you can download it from [. VS Code](#) also contains convenient features for interacting with `git`, but then the `git` command we describe below will need to be installed anyway.

You should get an account at some online `git` server provider as well, e.g. <https://github.com>.

Now, create a git repository `prog2` locally on your computer. Put your files in that directory and then add them to the repository as a commit. Finally, send the commit to the git server. You can use the VS Code git integration for some or all of these steps (see next section), or only use the terminal, as in the instructions below. **On your local computer:**

1. Navigate to your project directory by running (create first with `mkdir` if needed) `cd your_path`
2. Initialize git to set up your repository
`git init`
3. Add a folder or a file to the repository
`git add foldername`
4. Commit the files, i.e. save a snapshot of currently added changes to the version history with an associated description

```
git commit -m "Your commit message here"
```

5. Set up a remote repository on e.g. GitHub, initialize it as completely empty

```
git remote add origin https://github.com/yourusername/prog2.git
```

6. (Optional) Check the current branch name and change it to main if needed

```
git branch
```

```
git branch -M main
```

7. Push the files to the remote repository

```
git push origin main
```

On a department Linux server:

Once you are logged into a Linux machine, write

```
git clone https://github.com/yourusername/prog2.git
```

where the url is modified in both steps to fit your `git` server provider, username, and repository name. The provider tends to offer a prepared cloning url that you can just copy.

Note: If you use Github to handle your repository, it is required to use a so-called *Personal access token* to log in when you want to clone the repository (you can *not* use the password you use to login to the website). [Here](#) is a description of how to create an access token. In step 8 in the instructions for a “classic” access token, you just check “repo”.

Then you use the created access token instead of a password when `git` asks for it. To avoid having to enter this every time you access the repository, you can store it on the Linux machine. This can be achieved by doing the following once you have cloned the repository. Consider that this also means that whatever is accessible by the token will be accessible to anyone who is able to access your account on this machine:

```
abcde123@gullviva:~/prog2$ git config credential.helper store
abcde123@gullviva:~/prog2$ git pull
Username for 'https://github.com': abcdef12
Password for 'https://abcdef12@github.com':
Already up to date.
```

Now write (modify paths if they are different for you)

```
$ cd prog2/MA3/MA3_files
$ ls
```

and you should now see the files described in the previous section.

To test your code run (it is assumed below that `approximate_pi` has `print` statements):

```
$ python MA3.py
estimated pi for 1000 dots = 3.232
estimated pi for 10000 dots = 3.1344
estimated pi for 100000 dots = 3.14596
```

You can also test the following: Add `#!/usr/bin/env python3` in the beginning of your file. This is called a Shebang. Read more about Shebang <https://realpython.com/python-shebang/>.

Now you can run Python programs from the terminal without specifying an interpreter, if the file also gets execution permission. In a terminal:

```
$ ./MA3.py
$ ls -la
$ chmod 755 MA3.py
$ ls -la
$ ./MA3.py
```

The first time you run `$ ./MA3.py` you get an error message. Note the difference between the first and second time you execute `$ ls -la` for the file `MA3.py`. The command `chmod` changes the rights of the file, and `755` makes the file executable and readable by everyone and writable by you, represented by `ls` as `rw-r-xr-x`. When you have done this once you can execute the code by just typing `$ ./MA3.py` (as long as the permissions are unchanged).

After changes to the remote repository, maybe made using `git push` from another machine, you can update your local version by writing:

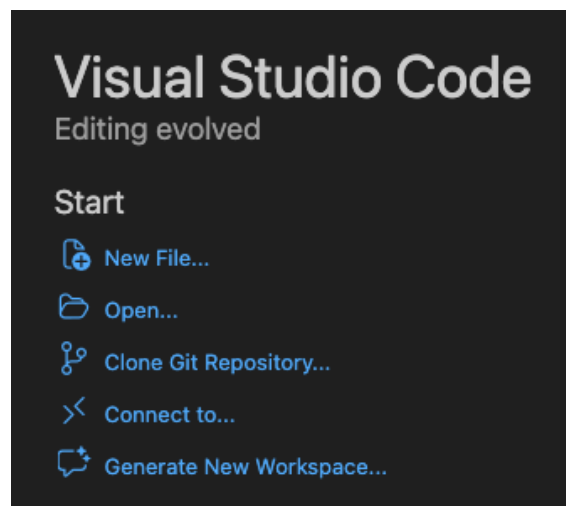
```
git pull --rebase origin main
```

2.3 Graphical interface

Remote development via SSH is available in VS Code with a graphical interface: <https://code.visualstudio.com/docs/remote/ssh>

Directly from the launch page, you can clone a git repository and connect to a server.

If some of the remote options are missing you might need to install the Remote - SSH extension. You can do that from the extension pane which you reach from the corresponding icon on the left-hand side.

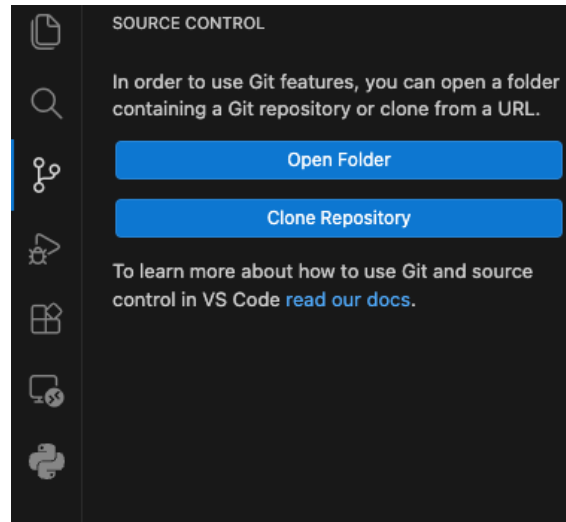


For connecting

- Click "Connect to". Alternatively,
 - Click "Remote explorer" in the left-side menu
 - Hover over SSH, click on the plus sign that appears (for creating a new remote)
- type `username@servername.it.uu.se`
- Connect by clicking on the arrow to the right of server name in the left menu

- Enter your password
- Note that you may need to specify the path to the interpreter: `/usr/bin/python`
- Now you are connected to a remote server and you can open your home folder there and proceed as usual.

You can clone a repository and then push and pull changes using Source Control from the left-side menu (see screenshot below)



If you do not want to work with git repositories from the command line, you might also want to try Github Desktop.

<https://desktop.github.com/download/>